

Large Scale and Multi-Structure Databases

Alice Orlandini

Indice

1	Introduzione	1
1.1	Big Data	1
1.1.1	Paradigma MapReduce	2
1.1.2	Hadoop	2
1.1.3	Spark	2
1.2	La rivoluzione dei database	3
1.2.1	Prima rivoluzione	3
1.2.2	Seconda rivoluzione	3
1.2.3	Terza rivoluzione	3
1.3	Transazioni ACID vs BASE	3
1.4	Data Persistency e Data Consistency	4
1.5	Il CAP Theorem	4
1.6	Eventual Consistency	5
2	Key Value DB	5
2.1	Introduzione	5
2.1.1	Design del Key Value DB	6
2.2	Hashing per la selezione dei server	6
2.3	Punti di forza e limitazioni	6
2.4	Data Compression	7
2.5	Data Partition	7
2.6	Replication	7
2.7	Hash Mapping	8
3	Document DB	8
3.1	Introduzione	8
3.1.1	Differenze tra XML e JSON	8
3.2	JSON Database	9
3.2.1	Modellazione dei dati	9
3.2.2	Modellazione delle gerarchie	10
3.3	Pianificazione di documenti mutabili	10
3.4	Normalizzazione e Denormalizzazione	10
3.5	Indexing	10
3.6	Sharding	11

4	Column DB	11
4.1	Introduzione	11
4.2	Differenze con i database relazionali	12
4.3	Delta Store	12
4.4	Projections	13
4.5	Hybrid Columnar Compression	13
4.6	Big Table	13
4.7	Column Families	13
4.8	KeySpace	14
4.9	Cluster e Partizioni	14
4.10	Architetture	14
4.11	Commit Logs	15
4.12	Bloom Filters	15
4.13	Consistency Level e Repliche	16
4.14	Anti-Entropy	16
4.15	Protocolli di Comunicazione	16
4.16	Cassandra	17
4.17	Indexing	17
4.18	Indici e Operazioni di Scrittura	18
5	Graph DB	18
5.1	Introduzione	18
5.2	Query di Mappatura	19
5.3	Scalabilità dei Graph DB	19
6	In-Memory DB	19
6.1	Introduzione	19
6.2	Soluzioni per la persistenza dei dati	20
6.3	TimesTen	20
6.4	Redis	20
6.5	HANA DB	20
6.6	Oracle 12c	21
7	Linee guida per la selezione di un database	21
7.1	Key-Value DB	22
7.2	Document DB	22
7.3	Column DB	22
7.4	Graph DB	23
7.5	Collaborazione	23

1 Introduzione

1.1 Big Data

I **Big Data** sono dati il cui volume, la cui diversità e la cui complessità richiedono **nuove tecniche, algoritmi ed analisi** per recuperarli, archivarli ed elaborarli. I servizi su Internet generano dati *continuamente* ad *alta velocità* per cui, i problemi ai quali andiamo incontro, sono la raccolta (**fetch**), l'immagazzinamento (**store**) e l'elaborazione (**elaboration**) dei dati stessi, quindi conoscere le caratteristiche di diverse architetture di database è diventato fondamentale.

Più formalmente, si definiscono Big Data i dati con le seguenti caratteristiche:

- **Volume:** grandi quantità di dati.
- **Varietà:** formati diversi di dati provenienti o dalla stessa sorgente o da sorgenti diverse. Tali dati andranno "normalizzati".
- **Velocità:** i dati possono essere generati con velocità diverse. In qualsiasi applicazione avremo infatti momenti di elevata e momenti di ridotta attività.
- **Valore:** i dati devono avere un certo valore, l'estrazione di tale valore può permetterci ad esempio di monetizzare.
- **Veridicità:** i dati acquisiti devono essere validati al fine di evitare inconsistenze, dati mancanti o dati malevoli.

Ma come si lavora con questa grande quantità di dati? Sono possibili due approcci: **scale-up** e **scale-out**.

In un approccio di **scale-up**, si cerca di migliorare le prestazioni del sistema **aggiungendo risorse più potenti** a una singola macchina o nodo. Ad esempio, si fa scale-up quando si aggiunge più CPU, RAM o disco rigido a un server esistente. I vantaggi sono la **maggiore potenza di elaborazione** su una singola macchina e la **semplice implementazione** e gestione. Gli svantaggi invece sono il **costo elevato** poiché l'acquisto di hardware più potente può essere dispendioso e il **limite fisico** sulla scala massima raggiungibile.

In un approccio di **scale-out**, si gestisce l'aumento delle esigenze di elaborazione **aggiungendo più macchine** o nodi al sistema. Ad esempio, si fa scale-out quando si aggiungono nuovi server al cluster di elaborazione dei dati. I vantaggi sono la **maggiore flessibilità** nel gestire grandi volumi di dati e carichi di lavoro crescenti e i **costi iniziali inferiori** in quanto è possibile utilizzare hardware meno costoso. Gli svantaggi invece sono la **maggiore complessità** nella gestione di un cluster di macchine distribuite ed il fatto che la **scalabilità potrebbe non essere lineare**, a seconda del tipo di applicazione. La scelta tra **scale-up** e **scale-out** dipende dalle esigenze specifiche del progetto, dal budget disponibile e dalla natura dei dati e delle operazioni di elaborazione. In alcuni casi, una combinazione di entrambi gli approcci può essere adottata per ottenere il massimo vantaggio in termini di prestazioni, scalabilità e costi.

Sottolineiamo infine l'importanza della **località dei dati**. La località dei dati riveste un ruolo cruciale nell'ottimizzazione delle prestazioni delle applicazioni, consentendo un accesso più **rapido e efficiente** ai dati. Mantenere i **dati vicini ai nodi di elaborazione** riduce il traffico di rete, migliorando l'efficienza complessiva del sistema. Inoltre, la distribuzione geografica dei dati può contribuire alla **tolleranza ai guasti**, garantendo una maggiore resilienza e disponibilità delle risorse.

1.1.1 Paradigma MapReduce

Il paradigma **MapReduce** si basa sull'assunzione che **muovere il risultato di un calcolo è più economico che muovere calcolo e dati allo stesso tempo**.

Questo paradigma, introdotto da Google nel 2012 per affrontare problemi di elaborazione su larga scala, adotta la strategia *divide et impera*: La fase di mappatura può essere eseguita in parallelo per i diversi blocchi di dati (*map*), e successivamente, la fase di riduzione (*reduce*) può combinare i risultati parziali ottenuti durante la fase precedente.

Il vantaggio di questo paradigma risiede nel fatto che è possibile **distribuire il carico di lavoro** tra tutti i nodi della rete di calcolatori che saranno in numero sufficiente per la grandezza dei dati con cui si ha a che fare. Inoltre, i costi della scalabilità, nel caso in cui il carico di lavoro aumentasse, sono notevolmente ridotti perché è sufficiente aggiungere nuovi calcolatori alla rete.

1.1.2 Hadoop

Hadoop è un **framework open source** progettato per l'elaborazione **distribuita** di grandi set di dati su cluster di computer. Basato sul modello di programmazione **MapReduce**, Hadoop consente di suddividere un problema complesso in task più piccoli, mapparli su nodi di un cluster, elaborarli in parallelo, e quindi ridurre i risultati finali.

Il cuore di Hadoop è costituito da due componenti principali: il sistema di archiviazione distribuita **Hadoop Distributed File System (HDFS)** e il modello di programmazione **MapReduce**. HDFS divide i dati in blocchi e li distribuisce su diversi nodi del cluster, garantendo una **ridondanza** per la tolleranza ai guasti. MapReduce, invece, gestisce il processo di **elaborazione distribuita**, suddividendo il lavoro tra i nodi.

Hadoop offre un ecosistema di progetti correlati che estendono le sue funzionalità come **HBase** che offre soluzioni per l'archiviazione di dati strutturati su Hadoop.

Sebbene Hadoop sia stato inizialmente sviluppato per gestire grandi volumi di dati su cluster di computer, ha alcune limitazioni in termini di prestazioni per determinati tipi di carichi di lavoro. Con l'avvento di framework come **Apache Spark**, che offre prestazioni superiori grazie all'elaborazione in memoria, Hadoop ha in parte ceduto il passo in contesti che richiedono velocità ed efficienza maggiori. Tuttavia, Hadoop continua a essere ampiamente utilizzato e costituisce la base di molte piattaforme di big data.

1.1.3 Spark

Apache Spark è un framework open source progettato per l'**elaborazione distribuita** di dati su larga scala. A differenza di Hadoop, Spark è noto per le sue prestazioni superiori grazie **all'elaborazione in memoria** e alla capacità di gestire operazioni iterative in modo più efficiente.

Il modello di programmazione di Spark si basa su concetti chiave come i **Resilient Distributed Datasets (RDD)**, che sono **collezioni di oggetti** distribuiti in memoria su un cluster. Questo modello rende Spark particolarmente adatto per scenari che richiedono operazioni iterative, come algoritmi di machine learning.

La natura in memoria di Spark significa che può superare le prestazioni di Hadoop, soprattutto in situazioni in cui l'accesso rapido ai dati è essenziale. Tuttavia, è importante notare che Hadoop e Spark spesso vengono utilizzati insieme in scenari complessi, sfruttando le forze complementari dei due framework.

1.2 La rivoluzione dei database

Un database è un'**organizzazione ordinata di dati** quindi i primi database mai creati risalgono ai tempi dei primi dizionari.

1.2.1 Prima rivoluzione

I **DBMS** rappresentavano un nuovo livello che consentiva la separazione tra la **logica di gestione** del database ed il **livello applicativo**.

Le sue caratteristiche principali sono la **definizione di uno schema** per i dati da memorizzare e di un **percorso di accesso** per la navigazione tra i record. I problemi principali erano che questi database funzionavano esclusivamente sui sistemi **mainframe**, era possibile effettuare solo le query previste durante la fase iniziale di progettazione, era estremamente difficile aggiungere nuovi elementi ad un sistema già esistente ed era orientato alle operazioni **CRUD** (create, read, update e delete) per cui le query analitiche complesse richiedevano un'intensa attività di codifica.

1.2.2 Seconda rivoluzione

La seconda rivoluzione è caratterizzata dall'introduzione della **teoria relazionale** in cui si hanno tuple, relazioni, vincoli ed operazioni sulle relazioni. È qui che si inizia a parlare di **terza forma normale** ad indicare che *gli attributi non chiave devono dipendere dall'attributo chiave, tutta la chiave e nient'altro che la chiave*.

1.2.3 Terza rivoluzione

Infine, nella terza rivoluzione i database sono caratterizzati da **scalabilità, flessibilità, disponibilità e basso costo**. Per permettere ciò si va a denormalizzare i database, si rinuncia quindi ad uno schema rigoroso e si introduce ridondanza al fine di aumentare le prestazioni.

1.3 Transazioni ACID vs BASE

Le transazioni **ACID** (Atomicità, Coerenza, Isolamento, Durabilità) e **BASE** (Basically Available, Soft state, Eventually consistent) rappresentano due approcci distinti per gestire la coerenza e la disponibilità dei dati nei **sistemi distribuiti**.

Nelle transazioni **ACID** si ha:

- **Atomicità**: le transazioni sono atomiche, il che significa che vengono eseguite completamente o non vengono eseguite affatto. Non ci sono stati intermedi.
- **Coerenza**: dopo il completamento di una transazione, il sistema si trova in uno stato coerente, rispettando tutte le regole di integrità dei dati.
- **Isolamento**: le transazioni sono isolate le une dalle altre, garantendo che l'esecuzione simultanea di più transazioni non comprometta la coerenza dei dati.
- **Durabilità**: una volta completata una transazione, le sue modifiche sono permanenti e persistono anche in caso di guasti del sistema.

Nelle transazioni BASE invece si ha:

- **Basically Available:** il sistema garantisce una disponibilità di base, anche a costo della incoerenza momentanea. Ciò significa che il sistema è sempre in grado di rispondere, anche se i dati potrebbero essere in uno stato transitorio.
- **Soft state:** il sistema accetta uno stato transitorio in cui i dati possono cambiare nel tempo, e la coerenza può richiedere un po' di tempo per essere raggiunta.
- **Eventually consistent:** la coerenza dei dati viene raggiunta *nel tempo*, senza garantire un momento specifico in cui i dati saranno coerenti. Il sistema consente ai nodi di convergere gradualmente verso uno stato consistente.

In sintesi, mentre ACID mira a garantire **coerenza immediata** e **prevedibile** dei dati a scapito della disponibilità in caso di fallimenti, BASE adotta un approccio più rilassato, consentendo una **maggior disponibilità** e **tolleranza ai guasti** a costo di una coerenza che può richiedere del tempo per essere raggiunta. Nei database **NoSQL** non è detto che le transazioni ACID siano supportate. La maggior parte di questi database assicurano la **scalabilità**, **disponibilità** e **flessibilità** e **controllo dei costi**.

1.4 Data Persistency e Data Consistency

La **persistenza dei dati** si riferisce alla capacità di **mantenere i dati** in modo che sopravvivano oltre la terminazione di un'applicazione o la disconnessione di un sistema. Gli aspetti chiave della persistenza dei dati includono: **durabilità**, **archiviazione** e **recuperabilità**.

La **coerenza dei dati** si riferisce alla **conformità** dei dati ad un certo insieme di regole o vincoli definiti. Quando si parla di consistenza dei dati, si cercano di evitare situazioni in cui i dati sono in uno stato ambiguo o contraddittorio.

1.5 Il CAP Theorem

Il **CAP Theorem** (Consistency, Availability, Partition Tolerance) è un concetto fondamentale nell'ambito della progettazione di sistemi distribuiti. Formulato da Eric Brewer nel 2000, il teorema afferma che **in un sistema distribuito**, è impossibile garantire **contemporaneamente** tutti e tre i seguenti aspetti:

- **Consistency (Consistenza):** tutti i nodi di un sistema vedono gli stessi dati contemporaneamente. In termini più semplici, se una modifica viene apportata a un dato, tutti i nodi vedono immediatamente il cambiamento.
- **Availability (Disponibilità):** ogni richiesta riceve una risposta, senza garanzia di contenuto rispetto alla richiesta. Anche se alcuni nodi del sistema falliscono, il sistema continua a rispondere alle richieste provenienti da nodi non interessati ai guasti.
- **Partition Tolerance (Tolleranza alle Partizioni):** il sistema continua a funzionare anche se si verificano partizioni di rete o perdite di messaggi tra i nodi. In altre parole, il sistema può tollerare la perdita di comunicazione tra alcuni nodi senza crollare completamente.

Secondo il teorema CAP, un sistema distribuito può massimizzare al massimo due di questi tre aspetti, a scapito del terzo. Le possibili soluzioni sono:

- **CA:** un sistema che sacrifica la tolleranza alle partizioni per garantire coerenza e disponibilità. Questo approccio è adatto per sistemi in cui la coerenza dei dati è critica e la perdita di alcuni nodi è accettabile. La coerenza totale può influire sulle prestazioni (latenza) e sulla scalabilità. Applicazioni che utilizzano questa soluzione sono quelle bancarie e di finanza.

- **CP**: un sistema che sacrifica la disponibilità per garantire la coerenza e la tolleranza alle partizioni. Questo approccio è utile in scenari in cui la coerenza è prioritaria rispetto alla disponibilità. Applicazioni che utilizzano questa soluzione sono quelle di e-commerce.
- **AP**: un sistema che sacrifica la coerenza per garantire disponibilità e tolleranza alle partizioni. Questo approccio è adatto per sistemi in cui la disponibilità è fondamentale, anche a costo di eventuali inconsistenze temporanee nei dati. Applicazioni che utilizzano questa soluzione sono ad esempio gli ATM.

1.6 Eventual Consistency

L'**Eventual Consistency** è un modello di consistenza nei **sistemi distribuiti** che accetta la possibilità di avere stati divergenti temporanei tra i nodi, garantendo tuttavia che alla fine tutti i nodi convergeranno verso uno stato consistente. Quindi, nel corso del tempo, il sistema converge verso la coerenza, ma durante il periodo transitorio, gli utenti che accedono a diversi database possono osservare diverse versioni dei dati. Esistono diverse modalità di Eventual Consistency:

- **Read Consistency**: garantisce che una volta che un utente ha aggiornato un record, tutte le sue letture di quel record restituiranno il valore aggiornato.
- **Session Consistency**: garantisce la "coerenza di lettura delle tue scritture" durante una sessione. Se l'utente termina una sessione e ne avvia un'altra con lo stesso DBMS, non vi è alcuna garanzia che il server "ricorderà" le scritture effettuate dall'utente nella sessione precedente.
- **Monotonic Read Consistency**: garantisce che se un utente invia una query e vede un risultato, tutti gli utenti non vedranno mai una versione precedente del valore.
- **Monotonic Write Consistency**: garantisce che se un utente effettua diversi comandi di aggiornamento, questi verranno eseguiti nell'ordine in cui li ha impartiti.
- **Causal Consistency**: questa forma di coerenza si basa sulla relazione causale tra le operazioni. Garantisce che se un'operazione A ha causato un'operazione B, allora tutti i nodi avranno A prima di B. Tuttavia, non fornisce una coerenza globale immediata, ma solo una coerenza causale tra operazioni correlate.

TODO: esempi.

2 Key Value DB

2.1 Introduzione

La prima struttura NoSQL che studiamo è quella dei Key Value Databases in cui ogni **valore** è identificato univocamente tramite un identificatore sotto forma di **chiave**.

In questo tipo di database, le **chiavi** sono uniche all'interno dello specifico **namespace** contenuto nello specifico **bucket**.

Le caratteristiche principali di questo database sono:

- **Semplicità**: non è necessario definire uno schema per il database né il tipo di dato per ogni attributo, dobbiamo solo definire il bucket e le sue chiavi. Tuttavia, non controllerà che venga inserito il valore corretto.
- **Velocità**: il database può scrivere sul disco il valore aggiornato di recente, mentre il programma lato utente sta facendo altro. Queste operazioni diventano ancora più veloci se si utilizza una **cache**. In questo caso, quando la memoria si riempie, il database libererà alcuni record utilizzando ad esempio l'algoritmo **Last Recently Used (LRU)**.

- **Scalabilità:** è la capacità di un database di aggiungere o rimuovere server per accogliere il carico del sistema. Questa proprietà è favorita dal fatto che non esistono join né vincoli tra gli attributi quindi si può semplicemente aggiungere o togliere il server senza rimaneggiare i dati. Per la scalabilità esistono due approcci:

1. **Master-Slave Replication:** il master accetta letture e scritture e raccoglie tutte le modifiche dei dati e di aggiornarli su tutti gli slave. Gli slave possono solo rispondere alle richieste di lettura. Questa architettura è adatta nelle applicazioni con una crescente richiesta di operazioni di lettura.

Pro:

- l'architettura è molto semplice perché solo il master comunica con gli slaves.
- non c'è bisogno di coordinare le operazioni di lettura al fine di evitare i conflitti.

Contro:

- se aumenta il carico di scritture, il master rischia di collassare.
- se il master diventa fuori uso allora il nodo non può accettare nuove scritture. In questo caso, il sistema collassa o comunque perde la capacità di scrittura.

2. **Masterless Replication:** si ha una struttura ad anello tra i server, tutti i server possono accettare le richieste di lettura e scrittura e i server aiutano i propri vicini per propagare le informazioni. Il principale Pro è che non si ha un unico point-of-failure come nel caso precedente ed è orientato ad accogliere un numero crescente di scritture.

2.1.1 Design del Key Value DB

Nel fare il design di questo tipo di database il problema principale è quello di trovare un set di chiavi **uniche** all'interno del namespace. La scelta della chiave va presa responsabilmente in quanto è bene che essa **non cambi nel tempo**.

La struttura consigliata per la chiave è `EntityName:EntityID:EntityAttribute` ad esempio: `customer:143291:firstName`. Notiamo che non si possono effettuare query complesse ma questo non è un problema poiché se avessimo questa necessità probabilmente dovremmo rivalutare la scelta del database, orientandoci su un database relazionale.

A basso livello la stringa che rappresenta la chiave viene convertita in una **stringa hash**. Per fare questa operazione si utilizza una **funzione hash** che prende in input una stringa e restituisce un valore intero in formato esadecimale.

2.2 Hashing per la selezione dei server

L'obiettivo è quello di bilanciare il carico di scritture in un'architettura Masterless. Avremo N **server distribuiti** ad anello ai quali andiamo ad assegnare un **ID univoco**.

Possiamo quindi prendere la chiave, applicare la funzione hash, dividere il valore ricevuto per N facendo il modulo (in modo da ottenere un intero tra 0 e N) e utilizzare il risultato per interrogare il server identificato da tale chiave.

Attenzione che questo non significa che si ha una sola copia del dato infatti ci possono essere le repliche, ma il server che risponderà alla richiesta sarà sempre lo stesso.

2.3 Punti di forza e limitazioni

Le principali operazioni che si possono fare con i Key Value DB sono:

- Prelevare dati per chiave
- Impostare il valore relativo ad una chiave
- Eliminare valori per chiave

Per fare operazioni più complesse bisogna lavorare lato codice applicativo. Il punto di forza è che queste operazioni sono molto veloci in quanto sono ottimizzate per la cache del server.

Le limitazioni invece sono:

- L'unico modo per prelevare i valori è per chiave.
- Alcuni database KV non supportano le query che lavorano su intervalli.
- Non esiste un linguaggio di interrogazione standardizzato come avviene per esempio con l'SQL nei database relazionali.

2.4 Data Compression

I Key Value DB sono **memory intensive** nel senso che fanno un utilizzo intensivo della memoria. Questo significa che quando bisogna effettuare delle operazioni di lettura e scrittura dal disco il database diventa **lento**.

Per ottimizzare l'utilizzo della memoria persistente si utilizzano **algoritmi di compressione dati** che permettono un compromesso tra **velocità** di compressione e decompressione e la **grandezza** dei dati compressi.

2.5 Data Partition

Le partizioni di un insieme di dati (chiamate **shard** o **chunk**) possono essere gestite da nodi diversi di un sistema distribuito. L'obiettivo principale è quello di **bilanciare** uniformemente i carichi di scrittura e lettura tra i server del sistema. Inoltre, deve essere facile ribilanciare i dati in seguito all'aggiunta o alla rimozione di un nodo nel cluster.

Si definisce **Partition Key** la chiave che identifica una specifica partizione nel quale troveremo il valore di interesse.

Le caratteristiche di un cluster sono:

- i nodi sono **poco accoppiati** nel senso che tendono ad essere **indipendenti** con un **coordinamento minimo** con gli altri nodi del cluster.
- ogni nodo è il responsabile delle operazioni sulle proprie partizioni e invia periodicamente messaggi ai vicini per indicare lo stato funzionante.
- quando un nodo si guasta, gli altri nodi possono rispondere alle richieste al suo posto, e, quando esso tornerà in funzione, i gli altri nodi si occuperanno di aggiornarlo.

Per garantire queste caratteristiche si utilizza una **struttura ad anello** con una **funzione hash** che ritorna il nodo che deve gestire la richiesta. Inoltre, per garantire **l'avaiability** si replicano le partizioni sui vicini dei nodi.

Lo svantaggio di questo metodo è che aggiornare le repliche richiede tempo ed energie computazionali.

2.6 Replication

L'avaiability è garantita dall'uso delle **repliche**. Vediamo le principali soluzioni per implementare le repliche. Definiamo:

- N numero di repliche.

- W numero di repliche che devono essere aggiornate prima che la scrittura venga completata.
- R numero di copie da interrogare per leggere un set di dati.

Se $N = 3$, le soluzioni possibili sono 4:

1. Mantenere aggiornate sempre tutte le copie, $W = 3$ e $R = 1$. Avremo **scritture lente, letture veloci, sistema consistente**.
2. Mantenere un buon livello di consistenza andando ad aggiornare almeno una delle due repliche del dato, $W = 2$ e $R = 2$. Avremo **scritture più veloci** rispetto alla soluzione precedenti, **letture più lente** rispetto alla soluzione precedente, **sistema mediamente consistente**.
3. Scriviamo solo su una replica ma leggiamo da tutte, $W = 1$ e $R = N$. Avremo **scritture veloci, letture lente ma consistenti**.
4. Scrivere e leggere solo da una replica, $W = 1$ e $R = 1$. Avremo **scritture veloci, letture veloci, sistema inconsistente**.

La scelta dipende dalla situazione e dai requisiti funzionali e non funzionali.

2.7 Hash Mapping

Le **funzioni hash** trasformano una chiave in un valore esadecimale chiamato hash. La prima cosa da tenere a mente è che anche un piccolo cambiamento nella chiave potrebbe comportare un grande cambiamento nell'hash. Quindi, indipendentemente dalla somiglianza tra due chiavi, esse saranno distribuite uniformemente sull'insieme di tutti i possibili output della funzione.

Uno dei problemi più importanti è quello della **collisione**, ovvero due o più chiavi con la medesima hash. La soluzione a questo problema coinvolge puntatori ed alberi binari.

Un altro problema riguarda la **consistenza**. Ipotizziamo di avere una serie di server, se, per qualsiasi motivo, dovessimo scambiare due server o aggiungerne/rimuoverne uno, dovremmo modificare tutte le hash. Questa operazione si chiama **riallocazione** e richiede di interrompere il servizio per un certo periodo di tempo. Una soluzione a questo problema si chiama **Coherence Hashing**. Si utilizza un'organizzazione circolare e partizionata delle chiavi in cui ogni server è responsabile di una delle sezioni. Quindi, nel caso di aggiunta di un nuovo server dovremo ridistribuire solo le chiavi dei suoi vicini e non dovremo nemmeno cambiare le funzioni hash.

3 Document DB

3.1 Introduzione

I **Document DB** sono database NoSQL che memorizzano i dati come **documenti strutturati** come XML e JSON. Essi sono schema-less quindi garantiscono un buon livello di flessibilità permettendo allo stesso tempo di gestire strutture dati complesse. A differenza dei Key-Value DB è possibile eseguire operazioni più complesse, come interrogazioni e filtri ed inoltre alcuni Document DB consentono anche transazioni ACID.

3.1.1 Differenze tra XML e JSON

XML è acronimo di *eXtensible Markup Language*, è un **linguaggio di markup** in cui si specifica la struttura e il contenuto del documento. Si basa sui **tag** (come l'html) che possono contenere tutti i tipi di informazioni.

I vantaggi sono:

- un documento XML può essere trasmesso in modo efficiente e visualizzato in modo diverso su diversi software.
- questi documenti possono essere **modularizzati** in modo da riutilizzarne delle parti.

Gli svantaggi sono:

- i tag XML sono prolissi e ripetitivi quindi la quantità di memoria richiesta per memorizzare questi documenti è elevata rispetto ad altre alternative.
- questi documenti sono computazionalmente onerosi da analizzare.

Per ovviare a questi svantaggi si è passati ai JSON Documents. **JSON** è l'acronimo di *JavaScript Object Notation* e viene usato per formattare i dati. Un JSON Document è composto da una collezione di **name-value** in cui i value possono essere molto complicati.

Le somiglianze con l'XML sono che:

- sono leggibili dall'uomo e hanno una sintassi molto semplice.
- sono gerarchici e indipendenti dalla lingua.
- sono supportati dalle API dei principali linguaggi di programmazione.

Le differenze invece sono che:

- la sintassi è diversa.
- JSON è meno prolisso e include gli array.
- in JSON ci sono alcune parole che non possono essere utilizzate in quanto sono riservate in Javascript.

3.2 JSON Database

In questo tipo di database, il JSON Document è l'unità base di memorizzazione. Un documento può a sua volta contenere altri documenti consentendo una struttura gerarchica complessa.

Si definisce **collezione** o **bucket** un insieme di documenti che condividono uno scopo comune. Infatti, possiamo avere **schemi polimorfici**, ovvero documenti che contengono altri documenti completamente diversi in termini di contenuto ma che condividono lo scopo.

Anch'esso è **schemaless** quindi non è necessario definire la struttura dell'oggetto. Il vantaggio di questo approccio è l'elevata flessibilità mentre lo svantaggio è che il DBMS spesso non è in grado di applicare le regole basate sulla struttura dati (per esempio per controllare la presenza di alcuni campi obbligatori).

Da notare che i Document DB non forniscono generalmente operazioni di join.

3.2.1 Modellazione dei dati

Consideriamo l'entità *cliente* con associata una lista di entità *indirizzi*. Un modo di implementare questa associazione è quella **dell'embedded document** quindi l'entità *cliente* avrà un campo *indirizzo* con valore un vettore di entità *indirizzo*.

Consideriamo ora l'entità *studente* che può seguire uno o più corsi. Possiamo modellare questa situazione considerando due collezioni: la prima contiene i corsi con un campo *studentiInscritti* che contiene un vettore di identificatori degli studenti, e un'altra collezione contenente le informazioni personali degli studenti. In questo caso, bisogna fare molta attenzione alle operazioni di aggiornamento in quanto il DBMS non controllerà l'integrità referenziale.

3.2.2 Modellazione delle gerarchie

Per modellare delle strutture gerarchiche (fatte ad albero) abbiamo tre possibilità:

1. **Parent Reference:** utile se dobbiamo frequentemente mostrare un'istanza specifica di un oggetto e solo dopo mostrare la categoria generale. In questo caso, si inserisce nella collezione `prodotto` l'ID della categoria.
2. **Child Reference:** questa soluzione è utile se dobbiamo frequentemente ritirare il nodo figlio di una specifica istanza di un oggetto. In questo caso, si inserisce nella `collection` padre un vettore di ID dei figli.
3. **List of Ancestor:** questa soluzione ci permette di ritirare tutto il percorso di discendenti in un'unica operazione di lettura. Tuttavia, un cambiamento nella struttura dell'albero può comportare molte operazioni di scrittura, a seconda del livello in cui è avvenuta la modifica. In questo caso, in ogni collezione si inserisce un campo `discendenti` che contiene un vettore di ID.

3.3 Pianificazione di documenti mutabili

Quando un documento viene creato, il DBMS gli assegna una certa quantità di spazio. Se il documento cresce più dello spazio allocato, il DBMS deve riposizionarlo e liberare lo spazio precedentemente allocato. Queste operazioni influiscono negativamente sulle prestazioni del sistema.

Una semplice soluzione a questo problema è quella di **allocare spazio sufficiente** al momento della **creazione** del documento. Inoltre, è buona norma memorizzare diversi tipi di documenti nella stessa raccolta solo se si è certi che essi verranno usati frequentemente insieme. Infine, la struttura dei documenti deve dipendere dalle interrogazioni che intendiamo implementare.

3.4 Normalizzazione e Denormalizzazione

Il processo di **normalizzazione** aiuta ad evitare anomalie dei dati ma può causare problemi di prestazioni poiché per ritirare un dato potrebbero essere necessarie delle operazioni di join.

Se invece si sceglie di introdurre dei dati **denormalizzati** si possono introdurre ridondanze, aumentando le prestazioni, ma bisogna prestare attenzione ad eventuali anomalie tra i dati. La denormalizzazione favorisce il miglioramento delle operazioni di lettura soprattutto quando si utilizzano di indici.

3.5 Indexing

Per evitare la scansione dell'intero database, i principali DBMS per la gestione di Document DB (come ad esempio MongoDB) supportano la definizione di indici. Gli **indici** sono un **insieme strutturato di informazioni** che permettono di passare da un attributo a informazioni correlate ad esso. In generale, gli indici sono strutture di dati speciali che memorizzano una piccola partizione del file in una forma facile da consultare.

L'indice viene implementato in modo che memorizzi il valore di un campo specifico o di un insieme di campi, **ordinato in base al valore del campo**. Tale ordinamento supporta efficienti interrogazioni con filtro di uguaglianza e interrogazioni basate su intervalli.

Applicazioni di analisi sono generalmente ad alta intensità di lettura quindi la definizione di indici permette di accedere efficientemente al database. Ad esempio, si possono definire indici per recuperare facilmente documenti che descrivono oggetti relativi ad una specifica regione geografica o ad un tipo specifico.

Tuttavia, maggiore è il numero di indici adottati, maggiore è il tempo per completare un'operazione di scrittura poiché tutti gli indici devono essere aggiornati oppure creati. Quindi, ridurre il numero di indici è preferibile nelle applicazioni ad alta intensità di scrittura.

3.6 Sharding

Lo **sharding** o **partizionamento orizzontale** è il processo di suddivisione dei dati in blocchi. Ogni **shard** (blocco) è suddiviso in uno specifico **nodo** (server) di un **cluster**. Ogni nodo può contenere un solo shard e, in caso di replica dei dati, uno shard può essere ospitato in più nodi.

I vantaggi di effettuare lo sharding sono che:

- consente di gestire carichi pesanti all'aumentare degli utenti del sistema.
- i dati possono essere facilmente distribuiti su un numero variabile di server che, a seconda delle esigenze, possono essere aggiunti o rimossi.
- è più economico del ridimensionamento verticale.
- se viene utilizzato in combinazione con le repliche, garantisce l'availability del sistema e una risposta rapida.

Per implementare lo sharding, è necessario selezionare una **chiave di shard** e un **metodo di partizionamento**. Una **chiave di shard** è un uno o più campi, presenti in tutti i documenti di una collezione, utilizzati per separare i documenti.

Gli **algoritmi di partizione** più utilizzati sono:

- **Intervallo**: se per esempio tutti i campi della collezione hanno il campo data di creazione, quest'ultimo potrebbe essere usato per partizionare i documenti in frammenti mensili.
- **Hashing**: una funzione hash può essere utilizzata per determinare la posizione di un documento, è possibile anche usare il **coherence hashing**.
- **Elenco**: se ad esempio stiamo considerando una collezione di prodotti con diversi tipi, questi ultimi possono essere usati come chiave shard per allocare i documenti per tipologia in ogni server diverso.

4 Column DB

4.1 Introduzione

I database basati su colonne rappresentano una categoria di sistemi di gestione di database che **organizzano e archiviano i dati in modo verticale**, differenziandosi dai tradizionali database relazionali che utilizzano una struttura di archiviazione orizzontale. In un database a colonne, ciascuna colonna di dati viene memorizzata su disco separatamente, consentendo **un rapido accesso ad informazioni specifiche senza dover recuperare l'intera riga di dati**.

Questi database sono particolarmente efficaci per **operazioni analitiche** e query che coinvolgono **aggregazioni e analisi su grandi volumi di dati**. La struttura a colonne favorisce un'ottimizzazione dello spazio di archiviazione e una maggiore compressione dei dati, riducendo così i tempi di risposta e **migliorando le prestazioni complessive** del sistema.

Tuttavia, ha prestazioni scarse se si deve modificare una singola riga. Se invece si vuole leggere un insieme di righe, si utilizzano delle cache che contengono interi blocchi del disco per aumentare l'efficienza.

Esistono infatti due tipologie di architetture:

- **OLTP (Online Transaction Processing)**: architetture software orientate alla gestione delle transazioni ACID.
- **OLAP (Online Analytics Processing)**: architetture software orientate all'analisi interattiva e veloce dei dati. I Column DB sono nati in seguito alla crescente richiesta di architetture basate sull'analisi dei dati.

In particolare, OLTP ha le seguenti caratteristiche:

- Elevato volume di transazioni
- Dati normalizzati
- Molte tabelle
- Elaborazione rapida

Mentre OLAP le seguenti

- Analisi di un elevato volume di dati
- Interrogazioni lente
- Dati denormalizzati
- Meno tabelle

Una query del primo tipo è "chi ha mangiato una mela?" mentre del secondo tipo: "quante persone hanno mangiato una mela?". Quindi quando si utilizza un database relazionale, sicuramente staremo utilizzando transazioni OLTP.

4.2 Differenze con i database relazionali

Le principali differenze tra i column DB e i database relazionali sono:

- I database basati su colonne sono implementati come mappe sparse e multidimensionali.
- Le colonne possono variare tra le righe.
- Le colonne possono essere aggiunte dinamicamente.
- Non si utilizzano i join perché i dati sono denormalizzati per ridurre il numero di tabelle a più di una tabella.
- Si consiglia di utilizzare un "keyspace" separato per ogni applicazione.

4.3 Delta Store

Il problema più grande di questi database sono che l'architettura di base risulta incapace di gestire un flusso costante di modifiche a livello di riga. Queste modifiche sono necessarie ogni volta che il data warehouse deve fornire informazioni in tempo reale, aggiornate al minuto.

La soluzione al problema è il **Delta Store**, cioè una porzione del database che **risiede in memoria, non compressa e ottimizzata per modifiche di dati ad alta frequenza**. I dati presenti nel delta store vengono **periodicamente uniti con lo store principale** orientato alle colonne (in cui i dati sono altamente compressi). È importante notare che le query potrebbero necessitare di accedere sia al delta store che al column store per restituire risultati completi e precisi.

4.4 Projections

Un secondo problema risiede nel fatto che query complesse spesso necessitano di leggere combinazioni di dati delle colonne. In risposta a questo problema, alcuni database basati su colonne introducono il concetto di "proiezioni".

Le proiezioni rappresentano **aggregazioni o insiemi di colonne che vengono frequentemente utilizzate insieme in operazioni di query**. Per ottimizzare l'accesso e accelerare le prestazioni, le colonne appartenenti a specifiche proiezioni vengono archiviate insieme sul disco. Questo approccio mira a semplificare e velocizzare l'esecuzione delle interrogazioni, consentendo un **recupero più efficiente dei dati**.

L'utilizzo di proiezioni diventa particolarmente rilevante in scenari in cui certe combinazioni di colonne sono richieste frequentemente, migliorando notevolmente la velocità di risposta del sistema.

4.5 Hybrid Columnar Compression

L'**Hybrid Columnar Compression (HCC)** è una tecnologia avanzata di compressione utilizzata nei database colonnari per **ottimizzare lo spazio di archiviazione** e migliorare le prestazioni del sistema. Questa tecnica rappresenta una combinazione intelligente di diverse strategie di compressione, mirando a massimizzare la riduzione delle dimensioni dei dati senza compromettere eccessivamente le operazioni di accesso e interrogazione.

Nell'Hybrid Columnar Compression invece di comprimere l'intera riga di dati, come nei tradizionali database relazionali, l'HCC si concentra sulla **compressione delle colonne individuali**. Questo approccio sfrutta la similitudine tra i valori nelle colonne, riducendo notevolmente lo spazio richiesto per memorizzare i dati.

4.6 Big Table

Big Table è stato uno dei primi column DB introdotto da Google nel 2006 per supportare Google Earth. Il motivo della sua introduzione è da ricercare nel fatto che Google aveva la necessità di gestire enormi quantità di dati, quindi necessitavano di **flexibility** ed **availability**.

In questo database se si vuole estrarre una riga, c'è bisogno di conoscere: l'id della riga, il nome della colonna e il timestamp. Quindi i dati hanno di fatto 3 indici.

Quando si effettua un'operazione, l'atomicità riguarda solo il fatto che le righe vengano mantenute ordinate.

4.7 Column Families

Le **Column Families** rappresentano un insieme di colonne che con alcune proprietà che vengono frequentemente utilizzate insieme. Ad esempio, due colonne che vengono utilizzate insieme potrebbero essere quella delle informazioni del cliente e quella dell'indirizzo. Queste due colonne faranno parte della stessa column family e verranno memorizzate sul disco vicine in modo da essere utilizzate efficientemente insieme.

Le righe in questo tipo di organizzazione vengono generalmente **ordinate** e **versionate**. Inoltre, le colonne in una famiglia possono essere modificate **dinamicamente**, ovvero non c'è uno schema predefinito come avviene nei database relazionali.

Quindi, per estrarre un dato singolo da un column db necessitiamo di:

- **RowID**: che è una sorta di chiave primaria.

- **Column Name:** che identifica la colonna.
- **Timestamp:** è necessario in modo da ritirare una specifica versione del dato.

Il punto di forza di una column family quindi è la possibilità di ordinare e organizzare i valori, di avere un controllo delle versioni grazie ai timestamp e di poter modificare e aggiungere colonne dal lato dell'applicazione senza problemi.

4.8 KeySpace

In genere quando si lavora con questa tipologia di database necessitiamo di un KeySpace che è una sorta di contenitore ad alto livello che identifica l'applicazione che stiamo considerando.

4.9 Cluster e Partizioni

Al fine di distribuire i dati si possono utilizzare i **Cluster** che garantiscono l'availability e la scalability utilizzando soprattutto le repliche. Invece, per risolvere il problema della consistency è possibile usare l'eventual consistency.

Inoltre, al fine di bilanciare il carico di lavoro si possono utilizzare le **partizioni**, che costituisce un sottoinsieme logico di un database. Queste ultime vengono solitamente utilizzate per memorizzare un insieme di dati in base a qualche attributo dei dati, e possono essere generate sulla base di:

- Un **intervallo di valori**, come il valore dell'ID di una riga.
- Un **valore hash**, come il valore hash del nome della colonna o dell'ID, o del nome della famiglia della colonna
- Un **elenco di valori**, come i nomi degli stati o delle province.
- Un composto di due o più delle opzioni precedenti.

Utilizzando le partizioni, la cosa più importante da garantire è che non dobbiamo accedere a server diversi per prelevare informazioni dalla stessa column family.

4.10 Architetture

Le due possibili architetture sono: la prima è **HBase**, la versione Apache di Big Table basata su Hadoop quindi basata sul paradigma master-slave, la seconda è **zookeeper** che è un nodo che coordina le comunicazioni tra tutti gli altri nodi.

La differenza fondamentale è che HBase è caratterizzato dal fatto che abbiamo diversi server e ognuno è specializzato in una funzione specifica, di conseguenza si hanno più point of failure.

Dall'altro lato, abbiamo l'architettura peer-to-peer di Cassandra. Ogni nodo può eseguire lo stesso programma di altri nodi e ci possono anche essere nodi specializzati in alcuni servizi. Questa organizzazione è molto utile per lo scaling orizzontale, possiamo avere repliche, partizionamento dei dati su nodi diversi.

4.11 Commit Logs

I **Commit Logs** sono dei file che permettono di velocizzare le operazioni di scrittura su un database. Infatti, quando si effettua un'operazione di scrittura su disco i dati potrebbero non essere posizionati adiacentemente e di conseguenza i tempi si allungano.

Per ovviare a ciò sono state introdotte questi file, chiamati commit logs, in cui è possibile solo accedere in append che, se utilizzati, assicurano che tutte le informazioni siano memorizzate nello stesso settore del disco. Quindi la scrittura nel commit log è più veloce della scrittura sulla famiglia di colonne specifica.

Quindi quello che succede è che:

- si effettua un'operazione di scrittura.
- si inserisce una voce nel commit logs che riporta tutti i dettagli dell'operazione di scrittura.
- si restituisce il controllo all'applicazione.
- prima o poi, un thread copierà le informazioni dal commit logs allo storage persistente.

Questi file vengono inoltre utilizzati per garantire un **elevato livello di controllo dei guasti**. Se durante l'aggiornamento del database si verifica un errore, i registri mostrano il punto in cui ci si trovava e si possono recuperare le informazioni eventualmente perse.

4.12 Bloom Filters

I **Bloom Filters** sono una struttura dati probabilistica, utilizzata come supporto per un'operazione di lettura.

Per eseguire un prelievo di un'informazione necessitiamo di una funzione get su un RowID, una colonna specifica e un timestamp, ma non siamo certi di trovare i dati nella posizione: ciò significa che potremmo effettuare un accesso al disco senza recuperare i dati. Per ridurre al minimo gli accessi inutili al disco, possiamo utilizzare questa struttura dati e le sue funzioni.

Funziona in questo modo:

1. si effettua una query verso la struttura dati relativa alla porzione di disco che vogliamo leggere;
2. la struttura può rispondere con "no" oppure "forse".
3. se la risposta è "no", allora siamo certi che i dati non si trovano nella parte di memoria che stiamo considerando.
4. se la risposta è "forse", potrebbe essere un falso positivo. I falsi negativi invece non sono ammessi.

Per implementare un Bloom Filter per una partizione al fine di sapere se uno specifico shard contiene o meno un dato possiamo costruire un array con N valori binari inizializzati a zero. Poi identifichiamo una collezione di $k \ll N$ funzioni hash utilizzate per mappare ogni chiave, ogni dato che stiamo cercando, in una posizione del vettore.

Ogni volta che effettuiamo un'operazione di scrittura, calcoliamo gli indici tramite una funzione hash che prende in input il RowID, il column name ed il timestamp, e impostiamo ad 1 i valori corrispondenti nell'array. Quando effettuiamo un'operazione di lettura, calcoliamo le k funzioni hash e accediamo all'array. In base al valore trovato, ricaviamo l'informazione che i dati sono presenti se e solo se tutti gli indici corrispondenti sono uguali a 1. Tuttavia, non siamo sicuri di trovare i dati in memoria, né che

tutti quanti i dati che stiamo cercando si trovino in quella porzione di memoria. Se invece anche solo uno dei valori trovati nell'array è a zero, siamo sicuri che il dato non si troverà lì.

Una proprietà importante dei Bloom Filters è che il tempo necessario sia per aggiungere un elemento sia per verificare se un elemento è presente nell'insieme è costante.

La probabilità di un falso positivo diminuisce all'aumentare di n , mentre aumenta con il numero di elementi memorizzati nel database, o se molti elementi vengono cancellati dal database.

Infine, la rimozione di un elemento dal Bloom Filter è impossibile perché i falsi negativi non sono ammessi quindi se si vuole eliminare bisogna resettare l'array.

Quindi per concludere, per velocizzare le operazioni di lettura si utilizzano i Bloom Filters, mentre per quelle di scrittura i Commit Logs.

4.13 Consistency Level e Repliche

Facciamo alcune considerazioni riguardo le repliche. I livelli di consistenza tra le repliche si dividono in:

1. **Strict Consistency:** tutte le repliche devono avere gli stessi dati.
2. **Low Consistency:** i dati devono essere scritti in almeno una delle repliche.
3. **Moderate Consistency:** soluzioni intermedie tra le precedenti.

Generalmente, nei column DB l'architettura logica è ad anello ed un server, identificato da una funzione hash funge da replica e il server primario si occupa di gestire la strategia di consistenza. In Cassandra le repliche sono memorizzate in nodi successivi negli anelli in senso orario.

4.14 Anti-Entropy

L'Anti-Entropy è il processo di rilevamento delle differenze nelle repliche. Il modo più semplice per implementarlo è controllare elemento per elemento se ci sono differenze, ma è molto pesante dal punto di vista computazionale. È infatti importante identificare le inconsistenze tra le repliche confrontando il più piccolo quantitativo di dati.

Un modo semplice per implementarlo è l'utilizzo di un **Hash Tree**: un albero binario le cui foglie contengono il valore hash dell'insieme di un blocco di dati.

Quindi, a ogni livello, abbiamo il valore hash degli hash dei suoi nodi figli. Quindi, possiamo calcolare il valore hash mettendo insieme gli hash dei due nodi sottostanti. Di conseguenza, la radice contiene il valore hash dell'intera partizione memorizzata. Si tratta quindi di una sorta di compressione delle informazioni del nostro database. Se si riscontrano differenze tra il vecchio albero e il nuovo albero realizzato sugli stessi dati presente in una replica, si può risalire a dove si trovano le differenze, partendo dalla radice e seguendo i diversi hash in discesa, fino a trovare la partizione in cui sono state identificate le differenze. In questo modo, trasferire l'albero ed esaminarlo è un processo veloce.

4.15 Protocolli di Comunicazione

Per implementare partizioni e repliche e aggiornarle, è fondamentale che i nodi siano a conoscenza delle condizioni degli altri. Il modo più semplice è quello di far parlare i nodi tra loro. Ciò funziona con pochi nodi, ma quando il numero di nodi aumenta, il numero di comunicazioni cresce in modo quadratico.

Sono stati quindi inventati tre approcci differenti:

- **Approccio Centralizzato:** si ha un server centrale che permette ai nodi di scambiarsi informazioni. Questo approccio ha il server come unico point of failure.

- **Approccio Totalmente Connesso:** tutti i nodi sono connessi agli altri quindi possono comunicare direttamente senza l'ausilio di un server intermedio. Questo approccio tuttavia non è scalabile.
- **Approccio basato su Gossip:** i nodi operano in parallelo e comunicano con altri nodi selezionati randomicamente.

4.16 Cassandra

In Cassandra, come detto si ha un'architettura ad anello in cui ogni nodo può gestire le richieste. Inoltre, ogni nodo sa dove sono contenute le informazioni che si stanno cercando e se un nodo è fuori servizio, altri possono sostituirlo.

Ad esempio, nel caso di operazioni di lettura, l'utente fa una richiesta, il nodo 1 risponde ma l'informazione che sta cercando si trova nel nodo 3. In questo caso, il nodo 1 inoltrerà la richiesta al nodo 3, che risponderà al nodo 1, e il nodo 1 risponderà all'utente.

Per le operazioni di scrittura avviene la stessa cosa: si fa una richiesta di scrittura, un nodo risponde e, se necessario, inoltra l'operazione di scrittura al nodo giusto. Se un nodo non è disponibile per qualsiasi motivo, il primo nodo mantiene la richiesta in un commit log.

Questo approccio si chiama **Hinted Handoff** ed è il processo che si occupa di memorizzare le informazioni delle operazioni di scrittura nei commit log, controllando periodicamente la disponibilità del server che deve essere aggiornato, e aggiornando le informazioni quando il server è disponibile.

4.17 Indexing

Un **Index** è una struttura dati speciale che permette all'engine del database di ritirare i dati più velocemente.

Nei Column DB esiste un indice primario costruito automaticamente sul RowID che rappresenta una sorta di chiave. Questo significa che le chiavi sono organizzate in modo da poter recuperare rapidamente la riga da parte del DBMS. Di solito l'utente può creare indici secondari su uno o più valori delle colonne.

La maggior parte dei DBMS consente di impostare gli indici ma alcuni non hanno questa funzione. Per ovviare a ciò possiamo implementare gli indici a livello applicativo. Di solito una regola di buon senso afferma che "se abbiamo bisogno di indici secondari sui valori delle colonne e il DBMS fornisce indici secondari gestiti automaticamente, allora dovremmo usare quelli".

La maggior parte dei DBMS utilizza per primo l'indice più selettivo. Ad esempio, in questa query:

```
SELECT name, surname FROM customers WHERE state = 'IT' AND surname = 'Smith';
```

Se abbiamo gli indici per gli stati e per i cognomi, selezioneremo prima l'indice per il quale abbiamo meno record e sarò il sistema a riconoscere automaticamente quello più selettivo.

Un altro vantaggio dell'uso degli indici secondari: immaginiamo di aver costruito l'applicazione seguendo i requisiti originali. Forse alcune query potrebbero essere aggiunte in seguito. La creazione di indici ci permette di migliorare le prestazioni senza dover manipolare nuovamente l'intero database.

Ci sono tuttavia alcune situazioni in cui è meglio evitare l'uso di indici gestiti automaticamente. Ad esempio:

- in una situazione in cui i valori che possono essere assunti da un attributo sono pochi e distribuiti in modo uniforme, come nel caso di un sì o di un no. In questo caso, probabilmente dovremo effettuare lo stesso numero di scansioni, con ulteriori problemi di ritardo causati dalla struttura dei dati;
- in una situazione in cui abbiamo troppi valori distinti, come un elenco di indirizzi e-mail, perché avremo una voce nell'indice per ogni voce del database;

- in una situazione in cui i valori associati a una colonna specifica sono troppo pochi e non vale la pena creare e gestire un'altra struttura di dati.

4.18 Indici e Operazioni di Scrittura

Quando effettuo un'operazione di scrittura devo aggiornare la tabella e la tabella dell'indice. Il primo modo per fare ciò è aggiornare la tabella, poi l'indice e quindi restituire il controllo all'applicazione. In questo caso generiamo un tempo aggiuntivo per concludere l'operazione di scrittura, ma siamo sicuri che tutto sia aggiornato.

Il secondo modo è quello di utilizzare gli **aggiornamenti in batch**, ovvero aggiornare le tabelle, restituire il controllo all'applicazione e fare un lavoro in batch per aggiornare la tabella degli indici. Questo introduce un eventual consistency, che in molti casi è accettabile.

L'atomicità dell'operazione di scrittura è garantita a livello di Column Family. Tuttavia, questa architettura non può essere utilizzata per le transazioni, perché i gruppi di righe non hanno l'atomicità garantita.

5 Graph DB

5.1 Introduzione

Si definisce **grafo** una collezione di vertici e archi. Ogni **vertice** rappresenta un'entità, ovvero istanze di classi (oggetti). Gli **archi** sono invece relazioni tra istanze di entità, non necessariamente dello stesso tipo, e possono avere delle proprietà associate. Gli archi possono avere una direzione o meno, in questo caso si parla di grafi **diretti** o **indiretti**. Se un'istanza interagisce con sé stessa allora si parla di **loop**.

Esistono applicazioni in cui si hanno più grafi che interagiscono tra loro, questi possono essere compattati tramite operazioni di **intersezione** o **unione**.

L'operazione di **attraversamento** di un grafo è quel processo che visita tutti i vertici di un grafo. Lo scopo è quello di poter effettuare operazioni di lettura o scrittura di una proprietà. L'attraversamento completo deve essere effettuato raramente essendo molto oneroso computazionalmente.

Due grafi si dicono **isomorfi** se:

- per ogni vertice del primo grafo esiste una corrispondenza nel secondo.
- per ogni arco tra una coppia di vertici del primo grafo esiste un arco corrispondente tra vertici del secondo grafo.

L'identificazione di sotto-grafi isomorfi permette di identificare pattern utili.

Un po' di nomenclatura:

- **Ordine**: numero di vertici.
- **Dimensione**: numero di archi.
- **Grado**: numero di archi connessi ad un vertice.
- **Closeness**: vicinanza, la misura di quanto lontano è il vertice da tutti gli altri vertici nel grafo.
- **Betweenness**: è la misura di quanto un arco fa da collo di bottiglia tra due vertici.

Le prime due misure sono importanti per valutare il **tempo** e la **l'occupazione di memoria** necessari per gestire applicazioni specifiche. Infatti, più alto è l'ordine e la dimensione del grafo, più è difficile risolvere le query.

La Closeness invece è utile per valutare la **velocità di propagazione delle informazioni** in una rete, partendo da un nodo specifico.

Le principali tipologie di grafo sono 3:

1. **Flow Network**: è un grafo direzionato caratterizzato dal fatto che in ogni nodo la somma dei pesi sugli archi entranti deve essere minore o uguale alla somma dei pesi degli archi uscenti. L'unica eccezione riguarda la sorgente.
2. **Bipartite Graph**: non è direzionato e abbiamo 2 set di vertici. Tra vertici dello stesso set non si possono effettuare relazioni, le uniche relazioni possono essere effettuate tra vertici appartenenti a set diversi. Può essere ad esempio usato per rappresentare la relazione tra studenti e insegnanti.
3. **Multigraph**: è un grafo in cui possiamo definire differenti tipi di archi tra vertici. Per esempio, da Lucca a Firenze posso andare in macchina e quindi avrò un certo tipo di relazione, oppure in treno e quindi avrò una seconda tipologia di relazione.

I Graph DB sono particolarmente utili quando la nostra applicazione richiede molte operazioni di join nel caso venisse realizzata con un database relazionale. Infatti, gli archi permettono di modellare esplicitamente le relazioni molti-a-molti, invece di usare le tabelle.

5.2 Query di Mappatura

Sono le query a guidare la progettazione di un Graph DB poiché ci permettono di individuare di quali entità e relazioni abbiamo bisogno.

Alcuni consigli per progettare al meglio questo tipo di database sono:

- prestare attenzione alle dimensioni del grafo, più è grande più la query sarà computazionalmente onerosa.
- se possibile, definire gli indici per migliorare i tempi di recupero delle informazioni.
- utilizzare un tipo di archi appropriato, in particolare se si ha a che fare con relazioni simmetriche utilizzare archi non diretti, altrimenti diretti. Inoltre, le proprietà richiedono memoria, quindi cercare di utilizzare solo le proprietà strettamente necessarie.
- fare attenzione ai cicli durante l'attraversamento del grafo.

5.3 Scalabilità dei Graph DB

La maggior parte di DBMS che gestiscono Graph DB non considerano la ripartizione tra i cluster né le repliche. Infatti, questi database sono progettati per essere eseguiti su un singolo server, per cui l'unica scalabilità possibile è quella **verticale**.

Quindi, bisogna prestare particolare attenzione alla crescita del grafo. Ad esempio, in un social network avremo utenti che si seguono l'un l'altro quindi all'aumentare degli utenti il grafo cresce sempre più.

6 In-Memory DB

6.1 Introduzione

Nel caso in cui la nostra applicazione non abbia bisogno di gestire grandi quantità di dati e di scalare, sia verticalmente che orizzontalmente, possiamo pensare di adottare i database In-Memory.

I database **In-Memory** vengono utilizzati quando tutti i dati dell'applicazione possono risiedere in memoria. Il loro utilizzo è stato favorito dalla diminuzione del prezzo per GB e dall'aumento del rapporto MB per chip.

Le sue caratteristiche principali sono:

- accesso molto rapido ai dati poiché si evitano lente operazioni di I/O verso i dispositivi di memorizzazione di massa.
- adatto alle applicazioni che non richiedono di scrivere continuamente su un supporto persistente.
- architettura ad hoc per l'implementazione della persistenza.
- architettura senza cache in quanto il database stesso è una cache.

6.2 Soluzioni per la persistenza dei dati

Essendo il database caricato completamente in memoria, bisogna capire come gestire eventuali guasti al sistema. Le principali soluzioni sono:

- **Replicare i dati ad altri membri del cluster:** essendo la memoria poco costosa, è un'operazione che si può fare.
- **Backup su disco:** si possono generare periodicamente degli snapshot del database da salvare su disco.
- **Adottare il registro delle transazioni:** si tiene un file di log in cui vengono riportate tutte le transazioni effettuate nel corso della giornata. Al termine, si effettuano le stesse operazioni sulla copia su disco. Il vantaggio è che il file di log occupa molto poco spazio.

6.3 TimesTen

TimesTen è un sistema di gestione dei dati in memoria fornito da **Oracle**. Si tratta di un database in memoria che consente di memorizzare e recuperare dati direttamente dalla RAM, offrendo prestazioni elevate e bassa latenza. Esso può essere utilizzato in situazioni in cui è fondamentale ottenere risposte in tempo reale dai dati.

La persistenza è mantenuta facendo degli snapshot su disco e dopo il commit delle transazioni. Per impostazione predefinita, **le scritture su disco sono asincrone**. Quindi, per garantire transazioni ACID, bisogna configurare il database per scrivere tutte le transazioni su un registro dopo ogni scrittura, con conseguente perdita di prestazioni.

6.4 Redis

Redis è un database, composto da dati chiave-valore, in memoria, noto per la sua velocità e flessibilità. Gli oggetti supportati sono stringhe, raccolte di stringhe come ad esempio le mappe hash.

È consentito ricercare solo per **chiave primaria** e non è consentito nessun meccanismo di indicizzazione secondaria.

Una caratteristica importante riguarda il fatto che Redis sfrutta un meccanismo di **memoria virtuale** per memorizzare una quantità di dati superiore alla memoria del server. Per fare ciò, le chiavi "vecchie" vengono scambiate con quelle nuove (algoritmo LRU), con conseguente perdita di prestazioni.

6.5 HANA DB

HANA DB è un sistema di gestione di database in memoria sviluppato da SAP. Esso combina un database relazionale con una soluzione di archiviazione colonnare.

Per performare al meglio, deve essere installato su una configurazione hardware ottimizzata.

Le sue tabelle colonna sono spesso utilizzate per operazioni **OLAP**. Per impostazione predefinita, vengono caricate in memoria su richiesta, tuttavia è possibile configurare il database per caricare dall'inizio un insieme di colonne.

6.6 Oracle 12c

Oracle 12c è una versione del sistema di gestione di database Oracle che introduce diverse caratteristiche innovative. La "c" in 12c sta per **Cloud**, sottolineando l'orientamento verso il **cloud computing**. Questa versione include miglioramenti significativi come il concetto di **database plug-in**, che consente di gestire più database come uno singolo, semplificando la gestione e migliorando l'efficienza delle risorse.

Oltre a offrire nuove funzionalità avanzate per la sicurezza e la gestione dei dati, Oracle 12c è stato progettato per supportare le esigenze di scalabilità e flessibilità richieste dai moderni ambienti di sviluppo e produzione dei database.

Nello specifico, si tratta di un modello relazionale in cui i dati vengono memorizzati per righe. Quindi i dati vengono mantenuti sul disco ma cachati in memoria.

7 Linee guida per la selezione di un database

I seguenti sono i passi da eseguire al fine di scegliere l'architettura più adatta alle esigenze dell'applicazione da implementare:

1. Richiesta dei requisiti al cliente.
2. Definizione dei requisiti funzionali e non funzionali.
3. Rappresentazione grafica dei casi d'uso (preferibilmente tramite diagramma UML).
4. Identificazione delle principali strutture dati e le relazioni tra di esse (preferibilmente tramite diagramma UML).
5. Progettazione del database.
6. Implementazione
7. Test
8. Manutenzione

L'ingegnere del software assume la responsabilità di identificare l'architettura ottimale per l'implementazione di software o hardware, definendo la struttura del database e selezionando il linguaggio di programmazione più appropriato.

I database relazionali vengono ancora utilizzati poiché sono loro i responsabili della protezione dell'integrità dei dati e di ridurre il rischio di perdite. Tuttavia, in un'era moderna le prestazioni e la disponibilità sono più importanti della coerenza e delle transazioni ACID.

Quindi, sulla base dei **requisiti funzionali**, che guidano la definizione delle query sul database e influenzano la natura delle relazioni tra i dati, possiamo scegliere tra:

- **DB Relazionali:** PostgreSQL o MySQL.
- **Key-Value DB:** Redis, Riak o DinamoDB.
- **Document DB:** MongoDB o CouchDB.
- **Column DB:** Cassandra o HBase.
- **Graph DB:** Neo4J o Titan.

I **requisiti non funzionali** vengono invece usati per definire:

- il **volume** di letture e scritture.
- la **tolleranza** per i dati incoerenti nelle repliche.
- requisiti di **disponibilità** e di **recovery** dai guasti.
- requisiti di **latenza**.

7.1 Key-Value DB

L'utilizzo di questi database è incoraggiato quando:

- l'applicazione deve gestire operazioni di lettura e scrittura **frequenti ma di piccola entità**.
- le strutture dati sono semplici e senza relazioni.
- sono necessarie query semplici, senza join o filtri.

Tuttavia, questi database non consentono un'eccessiva flessibilità e la possibilità di fare query su diversi attributi.

Alcuni esempi di applicazioni che possono sfruttare questo database sono:

- caching dei dati.
- carrello della spesa.
- memorizzazione di dati di configurazione.
- memorizzazione di oggetti di grandi dimensioni come immagini o file audio.

7.2 Document DB

L'utilizzo di questi database è incoraggiato quando:

- si ha necessità di memorizzare **attributi diversi** in termini di numero e tipologia.
- per memorizzare ed elaborare una **grande quantità** di dati.
- per effettuare query complesse utilizzando anche filtri e sfruttare l'indicizzazione.

La flessibilità è la sua caratteristica principale, insieme alle elevate prestazioni e alla facilità d'uso.

Alcuni esempi di applicazioni che possono sfruttare questo database sono:

- siti web con un elevato numero di letture e scritture.
- gestione di tipi di dati con attributi variabili, come ad esempio i prodotti di un e-commerce.
- applicazioni che utilizzano JSON come strutture dati.
- applicazioni che utilizzano la denormalizzazione per incorporare strutture all'interno di altre.

7.3 Column DB

L'utilizzo di questi database è incoraggiato quando:

- è richiesto di scrivere o leggere frequentemente.
- l'applicazione è distribuita geograficamente in più centri dati.
- l'applicazione può tollerare brevi inconsistenze nelle repliche.
- è richiesto di gestire campi dinamici.
- è richiesto di gestire enormi quantità di dati.

Questa tipologia di database è generalmente distribuita su più cluster. Se i dati gestiti dall'applicazione sono abbastanza piccoli per essere gestiti da un solo server, evitare questo database.

Alcuni esempi di applicazioni che possono sfruttare questo database sono:

- big science.
- analisi del mercato azionario.
- servizi di social network.

7.4 Graph DB

L'utilizzo di questi database è incoraggiato quando:

- l'applicazione si presta ad una rappresentazione a grafo.
- l'applicazione ha molte relazioni tra oggetti.
- è necessario attraversare rapidamente i percorsi tra entità.

Alcuni esempi di applicazioni che possono sfruttare questo database sono:

- gestione della rete di un'infrastruttura IT.
- gestione dei processi aziendali.
- raccomandazione di prodotti o servizi.
- social network.

7.5 Collaborazione

Notiamo che nello sviluppo di un'applicazione non è obbligatorio utilizzare un solo tipo di database ma possiamo **combinarli** e utilizzarli **contemporaneamente**.